objektinis_programavimas_2

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[detail level 1 2]

- c **Studentas**
  - c **CompareByLastName**
  - c **CompareByFinalGradeDesc**
- c **Zmogus**

objektinis_programavimas_2

benchmark.cpp File Reference

#include <iostream>
#include <iomanip>
#include <fstream>
#include <vector>
#include <list>
#include <deque>
#include <chrono>
#include <sstream>
#include <algorithm>
#include "**student.h**"

Go to the source code of this file.

Functions

int **main** ()

Function Documentation

◆ main()

int main ( )

Definition at line **13** of file **benchmark.cpp**.

objektinis_programavimas_2

benchmark.cpp

Go to the documentation of this file.

```
1  #include <iostream>
2  #include <iomanip>
3  #include <fstream>
4  #include <vector>
5  #include <list>
6  #include <deque>
7  #include <chrono>
8  #include <sstream>
9  #include <algorithm>
10  #include "student.h"
11
12
13  int main() {
14      std::vector<std::string> sizes = {"1k", "10k", "100k" };
15      std::vector<std::string> containers = {"vector", "list", "deque"};
16
17      std::cout << "\n";
18      std::cout << std::left
19                << std::setw(12) << "Container"
20                << std::setw(10) << "Strategy"
21                << std::setw(8) << "Size"
22                << std::setw(14) << "Partition(ms)"
23                << "\n";
24      std::cout << std::string(54, '-') << "\n";
25
26      for (const auto& container : containers) {
27          for (int strategy = 1; strategy <= 3; ++strategy) {
28              for (const auto& size : sizes) {
29                  std::string filename = size + ".txt";
30                  long long duration = -1;
31
32                  try {
33                      if (container == "vector") {
34                          duration = runPartitionBenchmark<std::vector<Mokinys>>(filename, strategy);
35                      } else if (container == "list") {
36                          duration = runPartitionBenchmark<std::list<Mokinys>>(filename, strategy);
37                      } else if (container == "deque") {
38                          duration = runPartitionBenchmark<std::deque<Mokinys>>(filename, strategy);
39                      }
40                  } catch (const std::exception& e) {
41                      std::cerr << "Error: " << e.what() << "\n";
42                  }
43
44                  std::cout << std::left
45                            << std::setw(12) << container
46                            << std::setw(10) << ("S" + std::to_string(strategy))
47                            << std::setw(8) << size
48                            << std::setw(14) << duration
49                            << "\n";
50              }
51          }
52      }
53
54      std::cout << "\nSummary:\n";
55      std::cout << "- Strategy 1: Copy to two containers (simple, uses more memory)\n";
56      std::cout << "- Strategy 2: Move and erase from original (slow for vector due to O(n) erase)\n";
57      std::cout << "- Strategy 3: STL stable_partition + assign (fastest for all containers)\n";
58
59      return 0;
60  }
```

objektinis_programavimas_2

Class Index

C | S | Z

C

S

Z

objektinis_programavimas_2

Studentas Member List

This is the complete list of members for **Studentas**, including all inherited members.

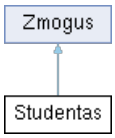
addIntermediateGrade(int grade)	**Studentas**	inline
calculateAverage() const	**Studentas**	inline private
calculateFinalGrade(const std::string &choice)	**Studentas**	inline
calculateMedian() const	**Studentas**	inline private
egz_rez	**Studentas**	private
galutinis	**Studentas**	private
getExamGrade() const	**Studentas**	inline
getFinalGrade() const override	**Studentas**	inline virtual
getFirstName() const	**Zmogus**	inline
getIntermediateGrades() const	**Studentas**	inline
getLastName() const	**Zmogus**	inline
operator!=(const Zmogus &other) const	**Zmogus**	inline
operator<(const Studentas &other) const	**Studentas**	inline
Zmogus::operator<(const Zmogus &other) const	**Zmogus**	inline
operator<<(std::ostream &out, const Studentas &studentas)	**Studentas**	friend
operator<=(const Studentas &other) const	**Studentas**	inline
Zmogus::operator<=(const Zmogus &other) const	**Zmogus**	inline
operator=(const Studentas &other)	**Studentas**	inline
operator=(Studentas &&other) noexcept	**Studentas**	inline
operator==(const Zmogus &other) const	**Zmogus**	inline
operator>(const Studentas &other) const	**Studentas**	inline
Zmogus::operator>(const Zmogus &other) const	**Zmogus**	inline
operator>=(const Studentas &other) const	**Studentas**	inline
Zmogus::operator>=(const Zmogus &other) const	**Zmogus**	inline
operator>>(std::istream &in, Studentas &studentas)	**Studentas**	friend
pavarde	**Zmogus**	protected
print(std::ostream &out) const override	**Studentas**	inline virtual
setExamGrade(int egz)	**Studentas**	inline
setFinalGrade(float gal)	**Studentas**	inline
setFirstName(const std::string &v)	**Zmogus**	inline
setLastName(const std::string &p)	**Zmogus**	inline
Studentas()	**Studentas**	inline
Studentas(const std::string &firstName, const std::string &lastName)	**Studentas**	inline
Studentas(const Studentas &other)	**Studentas**	inline
Studentas(Studentas &&other) noexcept	**Studentas**	inline
tarp_rez	**Studentas**	private
vardas	**Zmogus**	protected
Zmogus()	**Zmogus**	inline
Zmogus(const std::string &vardas, const std::string &pavarde)	**Zmogus**	inline
~Studentas()=default	**Studentas**	
~Zmogus()=default	**Zmogus**	virtual

objektinis_programavimas_2

Studentas Class Reference

#include <**student.h**>

Inheritance diagram for Studentas:



Classes

struct **CompareByLastName**

struct **CompareByFinalGradeDesc**

Public Member Functions

Studentas ()

Studentas (const std::string &firstName, const std::string &lastName)

~Studentas ()=default

Studentas (const **Studentas** &other)

Studentas (**Studentas** &&other) noexcept

Studentas & **operator=** (const **Studentas** &other)

Studentas & **operator=** (**Studentas** &&other) noexcept

const std::vector< int > & **getIntermediateGrades** () const

int **getExamGrade** () const

float **getFinalGrade** () const override

void **setExamGrade** (int egz)

void **setFinalGrade** (float gal)

void **addIntermediateGrade** (int grade)

bool **operator<** (const **Studentas** &other) const

bool **operator>** (const **Studentas** &other) const

bool **operator<=** (const **Studentas** &other) const

bool **operator>=** (const **Studentas** &other) const

void **calculateFinalGrade** (const std::string &choice)

void **print** (std::ostream &out) const override

Public Member Functions inherited from **Zmogus**

Private Member Functions

float **calculateAverage** () const

float **calculateMedian** () const

Private Attributes

std::vector< int > **tarp_rez**

int **egz_rez**

float **galutinis**

Friends

std::ostream & **operator<<** (std::ostream &out, const **Studentas** &studentas)

std::ostream & **operator<<** (std::ostream &out, const **Studentas** &studentas)

std::istream & **operator>>** (std::istream &in, **Studentas** &studentas)

Additional Inherited Members

Protected Attributes inherited from **Zmogus**

Detailed Description

Definition at line **64** of file **student.h**.

Constructor & Destructor Documentation

◆ Studentas() [1/4]

Studentas::Studentas ( ) inline

Definition at line **92** of file **student.h**.

◆ Studentas() [2/4]

Studentas::Studentas (const std::string & firstName,
const std::string & lastName) inline

Definition at line **94** of file **student.h**.

◆ ~Studentas()

Studentas::~Studentas ( ) default

◆ Studentas() [3/4]

Studentas::Studentas (const **Studentas** & other) inline

Definition at line **99** of file **student.h**.

◆ Studentas() [4/4]

Studentas::Studentas (**Studentas** && other) inline noexcept

Definition at line **105** of file **student.h**.

Member Function Documentation

◆ addIntermediateGrade()

void Studentas::addIntermediateGrade (int grade) inline

Definition at line **142** of file **student.h**.

◆ calculateAverage()

float Studentas::calculateAverage ( ) const

inline private

Definition at line **70** of file **student.h**.

◆ calculateFinalGrade()

void Studentas::calculateFinalGrade (const std::string & choice)

inline

Definition at line **172** of file **student.h**.

◆ calculateMedian()

float Studentas::calculateMedian ( ) const

inline private

Definition at line **79** of file **student.h**.

◆ getExamGrade()

int Studentas::getExamGrade ( ) const

inline

Definition at line **137** of file **student.h**.

◆ getFinalGrade()

float Studentas::getFinalGrade ( ) const

inline override virtual

Implements **Zmogus**.

Definition at line **138** of file **student.h**.

◆ getIntermediateGrades()

const std::vector< int > & Studentas::getIntermediateGrades ( ) const

inline

Definition at line **136** of file **student.h**.

◆ operator<()

bool Studentas::operator< (const **Studentas** & other) const

inline

Definition at line **144** of file **student.h**.

◆ operator<=()

bool Studentas::operator<= (const **Studentas** & other) const

inline

Definition at line **152** of file **student.h**.

◆ operator=() [1/2]

Studentas & Studentas::operator= (const **Studentas** & other) inline

Definition at line **114** of file **student.h**.

◆ operator=() [2/2]

Studentas & Studentas::operator= (**Studentas** && other) inline noexcept

Definition at line **124** of file **student.h**.

◆ operator>()

bool Studentas::operator> (const **Studentas** & other) const inline

Definition at line **148** of file **student.h**.

◆ operator>=()

bool Studentas::operator>= (const **Studentas** & other) const inline

Definition at line **156** of file **student.h**.

◆ print()

void Studentas::print (std::ostream & out) const inline override virtual

Reimplemented from **Zmogus**.

Definition at line **182** of file **student.h**.

◆ setExamGrade()

void Studentas::setExamGrade (int egz) inline

Definition at line **140** of file **student.h**.

◆ setFinalGrade()

void Studentas::setFinalGrade (float gal) inline

Definition at line **141** of file **student.h**.

Friends And Related Symbol Documentation

◆ operator<<

◆ operator<<

std::ostream & operator<< (std::ostream & out,
const **Studentas** & studentas) friend

Definition at line **199** of file **student.h**.

◆ operator>>

std::istream & operator>> (std::istream & in,
Studentas & studentas) friend

Definition at line **204** of file **student.h**.

Member Data Documentation

◆ egz_rez

int Studentas::egz_rez private

Definition at line **67** of file **student.h**.

◆ galutinis

float Studentas::galutinis private

Definition at line **68** of file **student.h**.

◆ tarp_rez

std::vector<int> Studentas::tarp_rez private

Definition at line **66** of file **student.h**.

The documentation for this class was generated from the following file:

- **student.h**

objektinis_programavimas_2

Zmogus Member List

This is the complete list of members for **Zmogus**, including all inherited members.

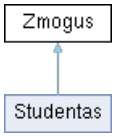
getFinalGrade() const =0	**Zmogus**	pure virtual
getFirstName() const	**Zmogus**	inline
getLastName() const	**Zmogus**	inline
operator!=(const Zmogus &other) const	**Zmogus**	inline
operator<(const Zmogus &other) const	**Zmogus**	inline
operator<=(const Zmogus &other) const	**Zmogus**	inline
operator==(const Zmogus &other) const	**Zmogus**	inline
operator>(const Zmogus &other) const	**Zmogus**	inline
operator>=(const Zmogus &other) const	**Zmogus**	inline
pavarde	**Zmogus**	protected
print(std::ostream &out) const	**Zmogus**	inline virtual
setFirstName(const std::string &v)	**Zmogus**	inline
setLastName(const std::string &p)	**Zmogus**	inline
vardas	**Zmogus**	protected
Zmogus()	**Zmogus**	inline
Zmogus(const std::string &vardas, const std::string &pavarde)	**Zmogus**	inline
~**Zmogus**()=default	**Zmogus**	virtual

objektinis_programavimas_2

Zmogus Class Reference abstract

#include <**student.h**>

Inheritance diagram for Zmogus:



Public Member Functions

	Zmogus ()
	Zmogus (const std::string &**vardas**, const std::string &**pavarde**)
virtual	**~Zmogus** ()=default
const std::string &	**getFirstName** () const
const std::string &	**getLastName** () const
void	**setFirstName** (const std::string &v)
void	**setLastName** (const std::string &p)
virtual float	**getFinalGrade** () const =0
virtual void	**print** (std::ostream &out) const
bool	**operator==** (const **Zmogus** &other) const
bool	**operator!=** (const **Zmogus** &other) const
bool	**operator<** (const **Zmogus** &other) const
bool	**operator<=** (const **Zmogus** &other) const
bool	**operator>** (const **Zmogus** &other) const
bool	**operator>=** (const **Zmogus** &other) const

Protected Attributes

std::string	**vardas**
std::string	**pavarde**

Detailed Description

Definition at line **14** of file **student.h**.

Constructor & Destructor Documentation

◆ Zmogus() [1/2]

Zmogus::Zmogus ( ) inline

Definition at line **20** of file **student.h**.

◆ Zmogus() [2/2]

Zmogus::Zmogus (const std::string & vardas,
const std::string & pavarde) inline

Definition at line **22** of file **student.h**.

◆ ~Zmogus()

virtual Zmogus::~Zmogus ( ) virtual default

Member Function Documentation

◆ getFinalGrade()

virtual float Zmogus::getFinalGrade ( ) const pure virtual

Implemented in **Studentas**.

◆ getFirstName()

const std::string & Zmogus::getFirstName ( ) const inline

Definition at line **27** of file **student.h**.

◆ getLastName()

const std::string & Zmogus::getLastName ( ) const inline

Definition at line **28** of file **student.h**.

◆ operator!=()

bool Zmogus::operator!= (const **Zmogus** & other) const inline

Definition at line **43** of file **student.h**.

◆ operator<()

bool Zmogus::operator< (const **Zmogus** & other) const inline

Definition at line **47** of file **student.h**.

◆ operator<=()

bool Zmogus::operator<= (const **Zmogus** & other) const inline

Definition at line **51** of file **student.h**.

◆ operator==()

bool Zmogus::operator== (const **Zmogus** & other) const inline

Definition at line **39** of file **student.h**.

◆ operator>()

bool Zmogus::operator> (const **Zmogus** & other) const inline

Definition at line **55** of file **student.h**.

◆ operator>=()

bool Zmogus::operator>= (const **Zmogus** & other) const inline

Definition at line **59** of file **student.h**.

◆ print()

virtual void Zmogus::print (std::ostream & out) const inline virtual

Reimplemented in **Studentas**.

Definition at line **35** of file **student.h**.

◆ setFirstName()

void Zmogus::setFirstName (const std::string & v) inline

Definition at line **30** of file **student.h**.

◆ setLastName()

void Zmogus::setLastName (const std::string & p) inline

Definition at line **31** of file **student.h**.

Member Data Documentation

◆ pavarde

std::string Zmogus::pavarde protected

Definition at line **17** of file **student.h**.

◆ vardas

std::string Zmogus::vardas protected

Definition at line **16** of file **student.h**.

The documentation for this class was generated from the following file:

- **student.h**

objektinis_programavimas_2

File List

Here is a list of all files with brief descriptions:

objektinis_programavimas_2

Here is a list of all functions with links to the classes they belong to:

- a -

- addIntermediateGrade() : **Studentas**

- c -

- calculateAverage() : **Studentas**
- calculateFinalGrade() : **Studentas**
- calculateMedian() : **Studentas**

- g -

- getExamGrade() : **Studentas**
- getFinalGrade() : **Studentas**, **Zmogus**
- getFirstName() : **Zmogus**
- getIntermediateGrades() : **Studentas**
- getLastName() : **Zmogus**

- o -

- operator!=() : **Zmogus**
- operator()() : **Studentas::CompareByFinalGradeDesc**, **Studentas::CompareByLastName**
- operator<() : **Studentas**, **Zmogus**
- operator<=() : **Studentas**, **Zmogus**
- operator=() : **Studentas**
- operator==() : **Zmogus**
- operator>() : **Studentas**, **Zmogus**
- operator>=() : **Studentas**, **Zmogus**

- p -

- print() : **Studentas**, **Zmogus**

- s -

- setExamGrade() : **Studentas**
- setFinalGrade() : **Studentas**
- setFirstName() : **Zmogus**
- setLastName() : **Zmogus**
- Studentas() : **Studentas**

- z -

- Zmogus() : **Zmogus**

- ~ -

- ~Studentas() : **Studentas**
- ~Zmogus() : **Zmogus**

objektinis_programavimas_2

Here is a list of all class members with links to the classes they belong to:

- a -

- addIntermediateGrade() : **Studentas**

- c -

- calculateAverage() : **Studentas**
- calculateFinalGrade() : **Studentas**
- calculateMedian() : **Studentas**

- e -

- egz_rez : **Studentas**

- g -

- galutinis : **Studentas**
- getExamGrade() : **Studentas**
- getFinalGrade() : **Studentas**, **Zmogus**
- getFirstName() : **Zmogus**
- getIntermediateGrades() : **Studentas**
- getLastName() : **Zmogus**

- o -

- operator!=() : **Zmogus**
- operator()() : **Studentas::CompareByFinalGradeDesc**, **Studentas::CompareByLastName**
- operator<() : **Studentas**, **Zmogus**
- operator<< : **Studentas**
- operator<=() : **Studentas**, **Zmogus**
- operator=() : **Studentas**
- operator==() : **Zmogus**
- operator>() : **Studentas**, **Zmogus**
- operator>=() : **Studentas**, **Zmogus**
- operator>> : **Studentas**

- p -

- pavarde : **Zmogus**
- print() : **Studentas**, **Zmogus**

- s -

- setExamGrade() : **Studentas**
- setFinalGrade() : **Studentas**
- setFirstName() : **Zmogus**
- setLastName() : **Zmogus**
- Studentas() : **Studentas**

- t -

- tarp_rez : **Studentas**

- v -

- vardas : **Zmogus**

- z -

- Zmogus() : **Zmogus**

- ~ -

- ~Studentas() : **Studentas**
- ~Zmogus() : **Zmogus**

objektinis_programavimas_2

Here is a list of all related symbols with links to the classes they belong to:

- operator<< : **Studentas**
- operator>> : **Studentas**

objektinis_programavimas_2

Here is a list of all variables with links to the classes they belong to:

- egz_rez : **Studentas**
- galutinis : **Studentas**
- pavarde : **Zmogus**
- tarp_rez : **Studentas**
- vardas : **Zmogus**

objektinis_programavimas_2

generator.cpp File Reference

```
#include <iostream>
#include <vector>
#include <list>
#include <deque>
#include <iomanip>
#include <fstream>
#include <random>
#include <stdexcept>
#include <chrono>
#include <sstream>
#include <algorithm>
#include "student.h"
```

Go to the source code of this file.

Functions

int **main** (int argc, char *argv[])

Function Documentation

◆ main()

```
int main ( int      argc,
           char * argv[] )
```

Definition at line **15** of file **generator.cpp**.

objektinis_programavimas_2

generator.cpp

Go to the documentation of this file.

```
1  #include <iostream>
2  #include <vector>
3  #include <list>
4  #include <deque>
5  #include <iomanip>
6  #include <fstream>
7  #include <random>
8  #include <stdexcept>
9  #include <chrono>
10  #include <sstream>
11  #include <algorithm>
12  #include "student.h"
13
14
15  int main(int argc, char* argv[]) {
16      std::vector<int> sizes = {1000, 10000, 100000, 1000000, 10000000};
17      std::vector<std::string> size_names = {"1k", "10k", "100k", "1M", "10M"};
18      std::vector<std::string> containers = {"vector", "list", "deque"};
19
20      std::cout << "Pradedami spartos tyrimai...\n";
21
22      for (size_t i = 0; i < sizes.size(); ++i) {
23          std::cout << "\n" << size_names[i] << ":\n";
24          runGenerationTestT<std::vector<Mokinys>>(size_names[i] + ".txt", sizes[i]);
25      }
26      for (const auto& container : containers) {
27          std::cout << "\n" << container << ":\n";
28          for (size_t i = 0; i < sizes.size(); ++i) {
29              std::string filename = size_names[i] + ".txt";
30              if (container == "vector") {
31                  runProcessingTest(filename);
32              } else if (container == "list") {
33                  runProcessingTest(filename);
34              } else if (container == "deque") {
35                  runProcessingTest(filename);
36              }
37          }
38      }
39      std::cout << "\nTyrimai baigti.\n";
40      return 0;
41  }
```

objektinis_programavimas_2

Here is a list of all functions with links to the files they belong to:

- calculateAverage() : **student.h**
- calculateFinalGrade() : **student.h**
- calculateMedian() : **student.h**
- displayResults() : **student.h**
- generateRandomData() : **student.h**
- generateRandomGrades() : **student.h**
- getPavardziuKiekis() : **student.h**
- getVarduKiekis() : **student.h**
- main() : **benchmark.cpp**, **generator.cpp**, **main.cpp**
- operator<<() : **student.h**
- operator>>() : **student.h**
- partitionStrategy1() : **student.h**
- partitionStrategy2() : **student.h**
- partitionStrategy3() : **student.h**
- printResults() : **student.h**
- readFromFile() : **student.h**
- readFromFileGeneric() : **student.h**
- readStudentData() : **student.h**
- runGenerationTest() : **student.h**
- runGenerationTestT() : **student.h**
- runPartitionBenchmark() : **student.h**
- runProcessingTest() : **student.h**
- writeResultsToAFile() : **student.h**

objektinis_programavimas_2

Here is a list of all file members with links to the files they belong to:

- calculateAverage() : **student.h**
- calculateFinalGrade() : **student.h**
- calculateMedian() : **student.h**
- displayResults() : **student.h**
- generateRandomData() : **student.h**
- generateRandomGrades() : **student.h**
- getPavardziuKiekis() : **student.h**
- getVarduKiekis() : **student.h**
- main() : **benchmark.cpp**, **generator.cpp**, **main.cpp**
- operator<<() : **student.h**
- operator>>() : **student.h**
- partitionStrategy1() : **student.h**
- partitionStrategy2() : **student.h**
- partitionStrategy3() : **student.h**
- pavardes : **student.h**
- printResults() : **student.h**
- readFromFile() : **student.h**
- readFromFileGeneric() : **student.h**
- readStudentData() : **student.h**
- runGenerationTest() : **student.h**
- runGenerationTestT() : **student.h**
- runPartitionBenchmark() : **student.h**
- runProcessingTest() : **student.h**
- vardai : **student.h**
- writeResultsToAFile() : **student.h**

objektinis_programavimas_2

Here is a list of all variables with links to the files they belong to:

- pavardes : **student.h**
- vardai : **student.h**

objektinis_programavimas_2

Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

[detail level 1 2]

- **Studentas::CompareByFinalGradeDesc**
- **Studentas::CompareByLastName**
- **Zmogus**
  - **Studentas**

objektinis_programavimas_2

objektinis_programavimas_2 Documentation

Generated by  1.16.1

objektinis_programavimas_2

main.cpp File Reference

#include <iostream>
#include <vector>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <stdexcept>
#include "**student.h**"

Go to the source code of this file.

Functions

int **main** ()

Function Documentation

◆ main()

int main ( )

Definition at line **9** of file **main.cpp**.

objektinis_programavimas_2

main.cpp

Go to the documentation of this file.

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <cstdlib>
5  #include <ctime>
6  #include <stdexcept>
7  #include "student.h"
8
9  int main() {
10     srand(static_cast<unsigned>(time(nullptr)));
11
12     int input_mode;
13     std::cout << "Pasirinkite įvesties būdą:\n"
14               << "1 - Rankinis įvedimas\n"
15               << "2 - Atsitiktinis generavimas\n"
16               << "3 - Nuskaitymas iš failo\n"
17               << "Jūsų pasirinkimas: ";
18     std::cin >> input_mode;
19
20     if (std::cin.fail() || input_mode < 1 || input_mode > 3) {
21         std::cout << "Neteisinga įvestis: tinka '1', '2' arba '3'.\n";
22         return 1;
23     }
24
25     std::vector<Studentas> students;
26
27     if (input_mode == 1) {
28         while (true) {
29             std::cout << "\nĮveskite " << students.size() + 1 << " studento duomenis:\n";
30             Studentas s;
31             readStudentData(s);
32             students.push_back(s);
33
34             std::string ans;
35             std::cout << "Ar norite įvesti dar vieną studentą? (t/n): ";
36             std::cin >> ans;
37             if (ans != "T" && ans != "t") {
38                 break;
39             }
40         }
41     }
42     else if (input_mode == 2) {
43         std::cout << "\nĮveskite skaiciu kiek studentu sugeneruoti:\n";
44         int mokSkaicius;
45         std::cin >> mokSkaicius;
46         for (int i = 0; i < mokSkaicius; i++) {
47             Studentas s;
48             generateRandomData(s);
49             students.push_back(s);
50         }
51         std::cout << "Sugeneruota " << mokSkaicius << " mokiniu ";
52     }
53     else {
54         std::string filename;
55         std::cout << "Įveskite failo pavadinimą: ";
56         std::cin >> filename;
57         students = readFromFile(filename);
58         if (students.empty()) {
59             std::cerr << "Nepavyko nuskaityti jokių duomenų iš failo. Programa baigiama." << std::endl;
60             return 1;
61         }
62         std::cout << "Iš failo nuskaityta " << students.size() << " studentų.\n";
63     }
64
65     std::string choice;
66     std::cout << "\nPasirinkite galutinio balo skaičiavimo būdą:\n"
67               << "1 - Vidurkis\n"
68               << "2 - Mediana\n"
69               << "Jūsų pasirinkimas: ";
70     std::cin >> choice;
71
72     if (choice != "1" && choice != "2") {
73         std::cout << "Neteisinga įvestis: tinka '1' arba '2'.\n";
74         return 1;
75     }
76
```

```
77      for (auto& s : students) {
78          calculateFinalGrade(s, choice);
79      }
80
81      int sort_choice;
82      std::cout << "\nPasirinkite rūšiavimo kriterijų:\n"
83                << "1 - Pagal vardą\n"
84                << "2 - Pagal pavardę\n"
85                << "3 - Pagal galutinį balą\n"
86                << "Jūsų pasirinkimas: ";
87      std::cin >> sort_choice;
88
89      if (std::cin.fail() || sort_choice < 1 || sort_choice > 3) {
90          std::cout << "Neteisinga įvestis: tinka '1', '2' arba '3'.\n";
91          return 1;
92      }
93
94      int output_option;
95      std::cout << "\nPasirinkite kur pateikti rezultatai:\n"
96                << "1 - i terminala\n"
97                << "2 - i faila\n"
98                << "Jūsų pasirinkimas: ";
99      std::cin >> output_option;
100
101      if (std::cin.fail() || output_option < 1 ||  output_option > 3) {
102          std::cout << "Neteisinga įvestis: tinka '1' arba '2'\n";
103          return 1;
104      }
105
106      if (sort_choice == 1) {
107          std::sort(students.begin(), students.end());
108      } else if (sort_choice == 2) {
109          std::sort(students.begin(), students.end(), Studentas::CompareByLastName());
110      } else {
111          std::sort(students.begin(), students.end(), Studentas::CompareByFinalGradeDesc());
112      }
113
114      if (output_option == 1) {
115          displayResults(students, choice);
116      } else {
117          writeResultsToAFile(students, choice, "output.txt");
118      }
119
120      return 0;
121  }
```

objektinis_programavimas_2

Studentas::CompareByFinalGradeDesc Member List

This is the complete list of members for **Studentas::CompareByFinalGradeDesc**, including all inherited members.

operator()(const Studentas &a, const Studentas &b) const **Studentas::CompareByFinalGradeDesc** inline

objektinis_programavimas_2

Studentas::CompareByFinalGradeDesc Struct Reference

#include <**student.h**>

Public Member Functions

bool **operator()** (const **Studentas** &a, const **Studentas** &b) const

Detailed Description

Definition at line **166** of file **student.h**.

Member Function Documentation

◆ operator()()

bool Studentas::CompareByFinalGradeDesc::operator() (const **Studentas** & a,
const **Studentas** & b) const inline

Definition at line **167** of file **student.h**.

The documentation for this struct was generated from the following file:

- **student.h**

objektinis_programavimas_2

Studentas::CompareByLastName Member List

This is the complete list of members for **Studentas::CompareByLastName**, including all inherited members.

operator()(const Studentas &a, const Studentas &b) const	**Studentas::CompareByLastName**	inline

objektinis_programavimas_2

Studentas::CompareByLastName Struct Reference

#include <**student.h**>

Public Member Functions

bool **operator()** (const **Studentas** &a, const **Studentas** &b) const

Detailed Description

Definition at line **160** of file **student.h**.

Member Function Documentation

◆ operator()()

bool Studentas::CompareByLastName::operator() (const **Studentas** & a,
const **Studentas** & b) const inline

Definition at line **161** of file **student.h**.

The documentation for this struct was generated from the following file:

- **student.h**

objektinis_programavimas_2

student.h File Reference

#include <iostream>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <chrono>
#include <cstring>

Go to the source code of this file.

Classes

class	**Zmogus**
class	**Studentas**
struct	**Studentas::CompareByLastName**
struct	**Studentas::CompareByFinalGradeDesc**

Functions

std::ostream &	**operator<<** (std::ostream &out, const **Studentas** &studentas)
std::istream &	**operator>>** (std::istream &in, **Studentas** &studentas)
int	**getVarduKiekis** ()
int	**getPavardziuKiekis** ()
void	**generateRandomGrades** (**Studentas** &studentas)
float	**calculateAverage** (const std::vector< int > &arr)
float	**calculateMedian** (std::vector< int > arr)
void	**generateRandomData** (**Studentas** &studentas)
void	**readStudentData** (**Studentas** &studentas)
std::vector< **Studentas** >	**readFromFile** (const std::string &filename)
void	**calculateFinalGrade** (**Studentas** &studentas, const std::string &choice)
void	**printResults** (const std::vector< **Studentas** > &students, const std::string &choice, std::ostream &out)
void	**displayResults** (const std::vector< **Studentas** > &students, const std::string &choice)
void	**writeResultsToAFile** (const std::vector< **Studentas** > &students, const std::string &choice, const std::string &filename)
void	**runGenerationTest** (const std::string &filename, int count)
template<typename Container>	
void	**runGenerationTestT** (const std::string &filename, int count)
void	**runProcessingTest** (const std::string &filename)
template<typename Container>	
Container	**readFromFileGeneric** (const std::string &filename)
template<typename Container>	
void	**partitionStrategy1** (const Container &students, Container &vargsiukai, Container &kietiakiai)
template<typename Container>	
void	**partitionStrategy2** (Container &students, Container &vargsiukai)
template<typename Container>	
void	**partitionStrategy3** (Container &students, Container &vargsiukai, Container &kietiakiai)

template<typename Container>

long long **runPartitionBenchmark** (const std::string &filename, int strategy)

Variables

const std::string **vardai** [] = {"Jonas", "Petras", "Antanas", "Vytautas", "Kazys", "Juozas", "Algirdas", "Bronius", "Edmundas", "Rimantas"}

const std::string **pavardes** [] = {"Jonaitis", "Petraitis", "Antanaitis", "Vytautaitis", "Kazaitis", "Juozaitis", "Algirdaitis", "Bronaitis", "Edmundaitis", "Rimantaitis"}

Function Documentation

◆ calculateAverage()

float calculateAverage (const std::vector< int > & arr)

Definition at line **253** of file **student.h**.

◆ calculateFinalGrade()

void calculateFinalGrade (**Studentas** & studentas,
const std::string & choice)

Definition at line **366** of file **student.h**.

◆ calculateMedian()

float calculateMedian (std::vector< int > arr)

Definition at line **262** of file **student.h**.

◆ displayResults()

void displayResults (const std::vector< **Studentas** > & students,
const std::string & choice)

Definition at line **391** of file **student.h**.

◆ generateRandomData()

void generateRandomData (**Studentas** & studentas)

Definition at line **273** of file **student.h**.

◆ generateRandomGrades()

void generateRandomGrades (**Studentas** & studentas)

Definition at line **246** of file **student.h**.

◆ getPavardziuKiekis()

```
int getPavardziuKiekis ( )
```

Definition at line **244** of file **student.h**.

◆ getVarduKiekis()

```
int getVarduKiekis ( )
```

Definition at line **243** of file **student.h**.

◆ operator<<()

```
std::ostream & operator<< ( std::ostream &       out,
                            const Studentas & studentas )
```

Definition at line **199** of file **student.h**.

◆ operator>>()

```
std::istream & operator>> ( std::istream & in,
                            Studentas &  studentas )
```

Definition at line **204** of file **student.h**.

◆ partitionStrategy1()

```
template<typename Container>
void partitionStrategy1 ( const Container & students,
                          Container &       vargsiukai,
                          Container &       kietiakiai )
```

Definition at line **537** of file **student.h**.

◆ partitionStrategy2()

```
template<typename Container>
void partitionStrategy2 ( Container & students,
                          Container & vargsiukai )
```

Definition at line **545** of file **student.h**.

◆ partitionStrategy3()

```
template<typename Container>
void partitionStrategy3 ( Container & students,
                          Container & vargsiukai,
                          Container & kietiakiai )
```

Definition at line **557** of file **student.h**.

Definition at line **557** of file **student.h**.

◆ printResults()

void printResults (const std::vector< **Studentas** > & students,
const std::string & choice,
std::ostream & out)

Definition at line **370** of file **student.h**.

◆ readFromFile()

std::vector< **Studentas** > readFromFile (const std::string & filename)

Definition at line **330** of file **student.h**.

◆ readFromFileGeneric()

template<typename Container>
Container readFromFileGeneric (const std::string & filename)

Definition at line **509** of file **student.h**.

◆ readStudentData()

void readStudentData (**Studentas** & studentas)

Definition at line **284** of file **student.h**.

◆ runGenerationTest()

void runGenerationTest (const std::string & filename,
int count)

Definition at line **404** of file **student.h**.

◆ runGenerationTestT()

template<typename Container>
void runGenerationTestT (const std::string & filename,
int count)

Definition at line **438** of file **student.h**.

◆ runPartitionBenchmark()

template<typename Container>
long long runPartitionBenchmark (const std::string & filename,
int strategy)

Definition at line **505** of file **student.h**.

Definition at line **565** of file **student.h**.

◆ runProcessingTest()

void runProcessingTest (const std::string & filename)

Definition at line **461** of file **student.h**.

◆ writeResultsToAFile()

void writeResultsToAFile (const std::vector< **Studentas** > & students,
const std::string & choice,
const std::string & filename)

Definition at line **395** of file **student.h**.

Variable Documentation

◆ pavardes

const std::string pavardes[] = {"Jonaitis", "Petraitis", "Antanaitis", "Vytautaitis", "Kazaitis", "Juozaitis", "Algirdaitis", "Bronaitis", "Edmundaitis", "Rimantaitis"}

Definition at line **241** of file **student.h**.

◆ vardai

const std::string vardai[] = {"Jonas", "Petras", "Antanas", "Vytautas", "Kazys", "Juozas", "Algirdas", "Bronius", "Edmundas", "Rimantas"}

Definition at line **240** of file **student.h**.

objektinis_programavimas_2

student.h

Go to the documentation of this file.

```
1  #ifndef STUDENT_H
2  #define STUDENT_H
3
4  #include <iostream>
5  #include <vector>
6  #include <iomanip>
7  #include <algorithm>
8  #include <fstream>
9  #include <sstream>
10  #include <stdexcept>
11  #include <chrono>
12  #include <cstring>
13
14  class Zmogus {
15  protected:
16      std::string vardas;
17      std::string pavarde;
18
19  public:
20      Zmogus() : vardas(""), pavarde("") {}
21
22      Zmogus(const std::string& vardas, const std::string& pavarde)
23          : vardas(vardas), pavarde(pavarde) {}
24
25      virtual ~Zmogus() = default;
26
27      const std::string& getFirstName() const { return vardas; }
28      const std::string& getLastName() const { return pavarde; }
29
30      void setFirstName(const std::string& v) { vardas = v; }
31      void setLastName(const std::string& p) { pavarde = p; }
32
33      virtual float getFinalGrade() const = 0;
34
35      virtual void print(std::ostream& out) const {
36          out << vardas << " " << pavarde;
37      }
38
39      bool operator==(const Zmogus& other) const {
40          return vardas == other.vardas && pavarde == other.pavarde;
41      }
42
43      bool operator!=(const Zmogus& other) const {
44          return !(*this == other);
45      }
46
47      bool operator<(const Zmogus& other) const {
48          return vardas < other.vardas;
49      }
50
51      bool operator<=(const Zmogus& other) const {
52          return *this < other || *this == other;
53      }
54
55      bool operator>(const Zmogus& other) const {
56          return !(*this <= other);
57      }
58
59      bool operator>=(const Zmogus& other) const {
60          return !(*this < other);
61      }
62  };
63
64  class Studentas : public Zmogus {
65  private:
66      std::vector<int> tarp_rez;
67      int egz_rez;
68      float galutinis;
69
70      float calculateAverage() const {
71          if (tarp_rez.empty()) return 0.0f;
72          int sum = 0;
73          for (int val : tarp_rez) {
74              sum += val;
75          }
76          return static_cast<float>(sum) / static_cast<float>(tarp_rez.size());
```

```
77      }
78
79      float calculateMedian() const {
80          if (tarp_rez.empty()) return 0.0f;
81          std::vector<int> sorted = tarp_rez;
82          std::sort(sorted.begin(), sorted.end());
83          size_t size = sorted.size();
84          if (size % 2 == 0) {
85              return (sorted[size/2 - 1] + sorted[size/2]) / 2.0f;
86          } else {
87              return static_cast<float>(sorted[size/2]);
88          }
89      }
90
91  public:
92      Studentas() : Zmogus(), egz_rez(0), galutinis(-1.0f) {}
93
94      Studentas(const std::string& firstName, const std::string& lastName)
95          : Zmogus(firstName, lastName), egz_rez(0), galutinis(-1.0f) {}
96
97      ~Studentas() = default;
98
99      Studentas(const Studentas& other)
100         : Zmogus(other.vardas, other.pavarde),
101           tarp_rez(other.tarp_rez),
102           egz_rez(other.egz_rez),
103           galutinis(other.galutinis) {}
104
105     Studentas(Studentas&& other) noexcept
106         : Zmogus(std::move(other.vardas), std::move(other.pavarde)),
107           tarp_rez(std::move(other.tarp_rez)),
108           egz_rez(other.egz_rez),
109           galutinis(other.galutinis) {
110         other.egz_rez = 0;
111         other.galutinis = -1.0f;
112     }
113
114     Studentas& operator=(const Studentas& other) {
115         if (this != &other) {
116             Zmogus::operator=(other);
117             tarp_rez = other.tarp_rez;
118             egz_rez = other.egz_rez;
119             galutinis = other.galutinis;
120         }
121         return *this;
122     }
123
124     Studentas& operator=(Studentas&& other) noexcept {
125         if (this != &other) {
126             Zmogus::operator=(std::move(other));
127             tarp_rez = std::move(other.tarp_rez);
128             egz_rez = other.egz_rez;
129             galutinis = other.galutinis;
130             other.egz_rez = 0;
131             other.galutinis = -1.0f;
132         }
133         return *this;
134     }
135
136     const std::vector<int>& getIntermediateGrades() const { return tarp_rez; }
137     int getExamGrade() const { return egz_rez; }
138     float getFinalGrade() const override { return galutinis; }
139
140     void setExamGrade(int egz) { egz_rez = egz; }
141     void setFinalGrade(float gal) { galutinis = gal; }
142     void addIntermediateGrade(int grade) { tarp_rez.push_back(grade); }
143
144     bool operator<(const Studentas& other) const {
145         return vardas < other.vardas;
146     }
147
148     bool operator>(const Studentas& other) const {
149         return galutinis > other.galutinis;
150     }
151
152     bool operator<=(const Studentas& other) const {
153         return galutinis <= other.galutinis;
154     }
155
156     bool operator>=(const Studentas& other) const {
157         return galutinis >= other.galutinis;
158     }
159
160     struct CompareByLastName {
161         bool operator()(const Studentas& a, const Studentas& b) const {
162             return a.pavarde < b.pavarde;
163         }
```

```
    };

    struct CompareByFinalGradeDesc {
        bool operator()(const Studentas& a, const Studentas& b) const {
            return a.galutinis > b.galutinis;
        }
    };

    void calculateFinalGrade(const std::string& choice) {
        float tarp_rez_val;
        if (choice == "1") {
            tarp_rez_val = calculateAverage();
        } else {
            tarp_rez_val = calculateMedian();
        }
        galutinis = 0.6f * egz_rez + 0.4f * tarp_rez_val;
    }

    void print(std::ostream& out) const override {
        out << vardas << " " << pavarde;
        out << " Tarpiniai pazymiai: [";
        for (size_t i = 0; i < tarp_rez.size(); ++i) {
            if (i > 0) out << ", ";
            out << tarp_rez[i];
        }
        out << "] Egzaminas: " << egz_rez;
        if (galutinis >= 0.0f) {
            out << " Galutinis: " << std::fixed << std::setprecision(2) << galutinis;
        }
    }

    friend std::ostream& operator<<(std::ostream& out, const Studentas& studentas);
    friend std::istream& operator>>(std::istream& in, Studentas& studentas);
};

std::ostream& operator<<(std::ostream& out, const Studentas& studentas) {
    studentas.print(out);
    return out;
}

std::istream& operator>>(std::istream& in, Studentas& studentas) {
    std::cout << "Iveskite varda: ";
    in >> studentas.vardas;

    std::cout << "Iveskite pavarde: ";
    in >> studentas.pavarde;

    std::cout << "Iveskite tarpiniu pazymiu skaiciu: ";
    int tarp_count;
    in >> tarp_count;

    studentas.tarp_rez.clear();
    for (int i = 0; i < tarp_count; ++i) {
        std::cout << "Iveskite " << (i + 1) << "-aji pazymi: ";
        int grade;
        in >> grade;
        if (grade < 0 || grade > 10) {
            std::cout << "Neteisingas pazymys (turi buti 0-10), praleidziama.\n";
            continue;
        }
        studentas.tarp_rez.push_back(grade);
    }

    std::cout << "Iveskite egzamino rezultata (0-10): ";
    int egz;
    in >> egz;
    if (egz < 0 || egz > 10) {
        std::cout << "Neteisingas egzamino pazymys, nustatoma 0.\n";
        egz = 0;
    }
    studentas.egz_rez = egz;
    studentas.galutinis = -1.0f;

    return in;
}

const std::string vardai[] = {"Jonas", "Petras", "Antanas", "Vytautas", "Kazys", "Juozas", "Algirdas", "Bronius", "Edmundas", "Rimantas"};
const std::string pavardes[] = {"Jonaitis", "Petraitis", "Antanaitis", "Vytautaitis", "Kazaitis", "Juozaitis", "Algirdaitis", "Bronaitis", "Edmundaitis",
    "Rimantaitis"};

int getVarduKiekis() { return 10; }
int getPavardziuKiekis() { return 10; }

void generateRandomGrades(Studentas& studentas) {
    int tarp_count = rand() % 10 + 1;
    for (int i = 0; i < tarp_count; ++i) {
        studentas.addIntermediateGrade(rand() % 11);
    }
```

```
    }
}

float calculateAverage(const std::vector<int>& arr) {
    if (arr.empty()) return 0.0f;
    int sum = 0;
    for (int val : arr) {
        sum += val;
    }
    return static_cast<float>(sum) / static_cast<float>(arr.size());
}

float calculateMedian(std::vector<int> arr) {
    if (arr.empty()) return 0.0f;
    std::sort(arr.begin(), arr.end());
    size_t size = arr.size();
    if (size % 2 == 0) {
        return (arr[size/2 - 1] + arr[size/2]) / 2.0f;
    } else {
        return static_cast<float>(arr[size/2]);
    }
}

void generateRandomData(Studentas& studentas) {
    studentas.setFirstName(vardai[rand() % getVarduKiekis()]);
    studentas.setLastName(pavardes[rand() % getPavardziuKiekis()]);

    int tarp_count = rand() % 10 + 1;
    for (int i = 0; i < tarp_count; ++i) {
        studentas.addIntermediateGrade(rand() % 11);
    }
    studentas.setExamGrade(rand() % 11);
}

void readStudentData(Studentas& studentas) {
    std::string firstName, lastName;
    std::cout << "Iveskite varda: ";
    std::cin >> firstName;
    studentas.setFirstName(firstName);

    std::cout << "Iveskite pavarde: ";
    std::cin >> lastName;
    studentas.setLastName(lastName);

    while (true) {
        std::cout << "Iveskite " << studentas.getIntermediateGrades().size() + 1
                  << " tarpini rezultata (arba -1, jei baigete): ";
        int grade;
        if (!(std::cin >> grade)) {
            std::cin.clear();
            std::cin.ignore(10000, '\n');
            std::cout << "Neteisinga ivedis. Bandykite dar karta.\n";
            continue;
        }

        if (grade == -1) {
            break;
        }
        if (grade < 0 || grade > 10) {
            std::cout << "Rezultatas turi buti nuo 0 iki 10. Bandykite dar karta.\n";
            continue;
        }
        studentas.addIntermediateGrade(grade);
    }

    if (studentas.getIntermediateGrades().empty()) {
        std::cout << "Turite ivesti bent viena tarpini rezultata. Generuojami atsitiktiniai.\n";
        generateRandomGrades(studentas);
    }

    std::cout << "Iveskite egzamino rezultata: ";
    int egz;
    while (!(std::cin >> egz) || egz < 0 || egz > 10) {
        std::cin.clear();
        std::cin.ignore(10000, '\n');
        std::cout << "Neteisinga ivedis (0-10). Bandykite dar karta: ";
    }
    studentas.setExamGrade(egz);
}

std::vector<Studentas> readFromFile(const std::string& filename) {
    std::vector<Studentas> students;
    std::ifstream file(filename);
    if (!file.is_open()) {
        throw std::runtime_error("Klaida: nepavyko atidaryti failo " + filename);
    }

    std::string line;
```

```
    std::getline(file, line);

    while (std::getline(file, line)) {
        if (line.empty()) continue;

        std::istringstream iss(line);
        Studentas s;
        std::string firstName, lastName;
        if (!(iss >> firstName >> lastName)) continue;
        s.setFirstName(firstName);
        s.setLastName(lastName);

        int grade;
        for (int i = 0; i < 5; ++i) {
            if (iss >> grade) {
                s.addIntermediateGrade(grade);
            }
        }
        int egz;
        iss >> egz;
        s.setExamGrade(egz);
        students.push_back(s);
    }

    file.close();
    return students;
}

void calculateFinalGrade(Studentas& studentas, const std::string& choice) {
    studentas.calculateFinalGrade(choice);
}

void printResults(const std::vector<Studentas>& students,
                  const std::string& choice,
                  std::ostream& out)
{
    const int langelio_ilgis = 20;
    std::string kategorija = (choice == "1") ? "Galutinis (Vid.)" : "Galutinis (Med.)";

    out << std::left;
    out << std::setw(langelio_ilgis) << "Pavarde"
        << std::setw(langelio_ilgis) << "Vardas"
        << std::setw(langelio_ilgis) << kategorija << '\n';
    out << std::string(3 * langelio_ilgis, '-') << '\n';

    for (const auto& s : students) {
        out << std::setw(langelio_ilgis) << s.getLastName()
            << std::setw(langelio_ilgis) << s.getFirstName()
            << std::setw(langelio_ilgis) << std::fixed << std::setprecision(2) << s.getFinalGrade()
            << '\n';
    }
}

void displayResults(const std::vector<Studentas>& students, const std::string& choice) {
    printResults(students, choice, std::cout);
}

void writeResultsToAFile(const std::vector<Studentas>& students, const std::string& choice, const std::string& filename) {
    std::ofstream outFile(filename);
    if (!outFile) {
        std::cerr << "Failed to open the file.\n";
        return;
    }
    printResults(students, choice, outFile);
}

void runGenerationTest(const std::string& filename, int count) {
    auto start = std::chrono::high_resolution_clock::now();

    std::vector<Studentas> students;
    students.reserve(count);

    for (int j = 0; j < count; j++) {
        Studentas s;
        generateRandomData(s);
        students.push_back(s);
    }

    std::ofstream outFile(filename);
    outFile << std::left << std::setw(20) << "Vardas" << std::setw(20) << "Pavarde";
    for (int i = 0; i < 5; i++) {
        outFile << std::setw(10) << "Pazymys";
    }
    outFile << std::setw(10) << "Egzaminas\n";

    for (const auto& s : students) {
        outFile << std::left << std::setw(20) << s.getFirstName() << std::setw(20) << s.getLastName();
```

```
        for (int grade : s.getIntermediateGrades()) {
            outFile << std::setw(10) << grade;
        }
        outFile << std::setw(10) << s.getExamGrade() << "\n";
    }
    outFile.close();

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();
    std::cout << filename << " generated " << count << " students in " << duration << "ms\n";
}

template<typename Container>
void runGenerationTestT(const std::string& filename, int count) {
    auto start = std::chrono::high_resolution_clock::now();
    Container students;
    for (int j = 0; j < count; j++) {
        Studentas s;
        generateRandomData(s);
        students.push_back(s);
    }
    std::ofstream outFile(filename);
    outFile << std::left << std::setw(20) << "Vardas" << std::setw(20) << "Pavarde";
    for (int i = 0; i < 5; i++) outFile << std::setw(10) << "Pazymys";
    outFile << std::setw(10) << "Egzaminas\n";
    for (const auto& s : students) {
        outFile << std::left << std::setw(20) << s.getFirstName() << std::setw(20) << s.getLastName();
        for (int grade : s.getIntermediateGrades()) outFile << std::setw(10) << grade;
        outFile << std::setw(10) << s.getExamGrade() << "\n";
    }
    outFile.close();
    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();
    std::cout << filename << " generated " << count << " students in " << duration << "ms\n";
}

void runProcessingTest(const std::string& filename) {
    auto read_start = std::chrono::high_resolution_clock::now();
    std::vector<Studentas> students = readFromFile(filename);
    auto read_end = std::chrono::high_resolution_clock::now();
    auto read_duration = std::chrono::duration_cast<std::chrono::milliseconds>(read_end - read_start).count();
    std::cout << filename << " read " << read_duration << "ms\n";

    auto sort_start = std::chrono::high_resolution_clock::now();
    for (auto& s : students) {
        s.calculateFinalGrade("1");
    }

    std::vector<Studentas> vargsiukai;
    std::vector<Studentas> kietiakiai;

    for (const auto& s : students) {
        if (s.getFinalGrade() < 5.0f) {
            vargsiukai.push_back(s);
        } else {
            kietiakiai.push_back(s);
        }
    }
    auto sort_end = std::chrono::high_resolution_clock::now();
    auto sort_duration = std::chrono::duration_cast<std::chrono::milliseconds>(sort_end - sort_start).count();
    std::cout << filename << " sort " << sort_duration << "ms\n";

    auto write_start = std::chrono::high_resolution_clock::now();
    auto write_to_file = [&](const std::string& fname, const std::vector<Studentas>& list) {
        std::ofstream out(fname);
        out << std::left << std::setw(20) << "Vardas" << std::setw(20) << "Pavarde" << "Galutinis\n";
        for (const auto& s : list) {
            out << std::left << std::setw(20) << s.getFirstName() << std::setw(20) << s.getLastName()
                << std::fixed << std::setprecision(2) << s.getFinalGrade() << "\n";
        }
        out.close();
    };

    write_to_file("vargsiukai.txt", vargsiukai);
    write_to_file("kietiakai.txt", kietiakai);
    auto write_end = std::chrono::high_resolution_clock::now();
    auto write_duration = std::chrono::duration_cast<std::chrono::milliseconds>(write_end - write_start).count();
    std::cout << filename << " write " << write_duration << "ms\n";

    auto total_duration = std::chrono::duration_cast<std::chrono::milliseconds>(write_end - read_start).count();
    std::cout << filename << " all " << total_duration << "ms\n";
}

template<typename Container>
Container readFromFileGeneric(const std::string& filename) {
    Container students;
    std::ifstream file(filename);
```

```
512     if (!file.is_open()) throw std::runtime_error("Cannot open: " + filename);
513     std::string line;
514     std::getline(file, line);
515     while (std::getline(file, line)) {
516         if (line.empty()) continue;
517         std::istringstream iss(line);
518         Studentas s;
519         std::string firstName, lastName;
520         iss >> firstName >> lastName;
521         s.setFirstName(firstName);
522         s.setLastName(lastName);
523         int grade;
524         for (int i = 0; i < 5; ++i) {
525             if (iss >> grade) s.addIntermediateGrade(grade);
526         }
527         int egz;
528         iss >> egz;
529         s.setExamGrade(egz);
530         s.calculateFinalGrade("1");
531         students.push_back(s);
532     }
533     return students;
534 }
535
536 template<typename Container>
537 void partitionStrategy1(const Container& students, Container& vargsiukai, Container& kietiakiai) {
538     for (const auto& s : students) {
539         if (s.getFinalGrade() < 5.0f) vargsiukai.push_back(s);
540         else kietiakiai.push_back(s);
541     }
542 }
543
544 template<typename Container>
545 void partitionStrategy2(Container& students, Container& vargsiukai) {
546     for (auto it = students.begin(); it != students.end(); ) {
547         if (it->getFinalGrade() < 5.0f) {
548             vargsiukai.push_back(std::move(*it));
549             it = students.erase(it);
550         } else {
551             ++it;
552         }
553     }
554 }
555
556 template<typename Container>
557 void partitionStrategy3(Container& students, Container& vargsiukai, Container& kietiakiai) {
558     auto partition_it = std::stable_partition(students.begin(), students.end(),
559         [](const Studentas& s) { return s.getFinalGrade() >= 5.0f; });
560     kietiakiai.assign(students.begin(), partition_it);
561     vargsiukai.assign(partition_it, students.end());
562 }
563
564 template<typename Container>
565 long long runPartitionBenchmark(const std::string& filename, int strategy) {
566     Container students = readFromFileGeneric<Container>(filename);
567     for (auto& s : students) {
568         s.calculateFinalGrade("1");
569     }
570
571     auto part_start = std::chrono::high_resolution_clock::now();
572     Container vargsiukai, kietiakiai;
573
574     if (strategy == 1) {
575         partitionStrategy1(students, vargsiukai, kietiakiai);
576     } else if (strategy == 2) {
577         partitionStrategy2(students, vargsiukai);
578     } else {
579         partitionStrategy3(students, vargsiukai, kietiakiai);
580     }
581
582     auto part_end = std::chrono::high_resolution_clock::now();
583     return std::chrono::duration_cast<std::chrono::milliseconds>(part_end - part_start).count();
584 }
585
586 #endif
```